

MentorSync AI

Documento de estado del proyecto — arquitectura, decisiones y avance

Generado el 1 de septiembre de 2026

Proyecto académico SENA (ADSO) — Plataforma de mentoría en vivo con asistente de IA por curso mediante RAG.

Cómo usar este documento: si retomas el proyecto con Claude (u otra IA) y tus tokens/créditos se agotaron, pega o adjunta este PDF al inicio de la conversación nueva. Contiene todas las decisiones de arquitectura, convenciones de código y el estado real de avance, para que la nueva sesión tenga el mismo contexto sin que tengas que explicar todo de nuevo.

1. Qué es MentorSync AI

Plataforma web de mentoría híbrida (estilo Platzi, pero diferenciada) donde los mentores dictan cursos en vivo y cada curso tiene un asistente de IA (bot) entrenado por el propio mentor, que responde preguntas de los aprendices cuando el mentor no está presente. El bot se entrena subiendo un PDF con el contenido del curso (proceso RAG: el PDF se trocea, se convierte en embeddings y se guarda en MongoDB Atlas Vector Search).

El diferenciador frente a Platzi/Udemy no es el catálogo de cursos, es la continuidad de mentoría aun sin el mentor conectado.

Roles del sistema

Rol	Permisos previstos
aprendiz	Inscribirse a cursos, chatear en vivo, interactuar con el bot del curso
mentor	Crear cursos, subir PDF de entrenamiento, dictar sesiones, ver analítica del bot
administrador	Control total: aprobar mentores, moderar contenido, ver métricas globales

2. Stack tecnológico (fijo)

Capa	Tecnología
Frontend	Angular 22 (signal-first), estilo glassmorphism, Vitest
Backend	Node.js 24.19 + Express 4.21.2 (ESM, no CommonJS)
Base de datos	MongoDB Atlas, con Atlas Vector Search para el RAG
IA generativa (chat)	Groq API — modelos Llama, gratis y muy rápido

Embeddings (RAG)	@xenova/transformers local en Node (all-MiniLM-L6-v2, 384 dim) — Groq no tiene endpoint de embeddings
Tiempo real	Socket.io
Seguridad backend	helmet, express-mongo-sanitize, express-rate-limit, compression
Logs	pino + pino-http (JSON estructurado, no console.log/morgan)
Docs de API	swagger-jsdoc + swagger-ui-express en /api-docs
Render markdown del bot	marked + DOMPurify.sanitize() SIEMPRE antes de insertar en el DOM
Testing backend	Jest + Supertest + mongodb-memory-server
Testing frontend	Vitest (no Karma/Jasmine)
Gestor de paquetes	npm (fijado en frontend/package.json vía Corepack)

Nota: se evaluó y se descartó Express 5 por incompatibilidad de express-mongo-sanitize (req.query pasó a solo-lectura en Express 5). Se decidió quedarse en Express 4.21.2.

3. Arquitectura

Backend: arquitectura en capas (Layered Architecture) con Service Layer Pattern — Controller → Service → Repository → Model. Cada capa solo habla con la inmediatamente inferior; el controller nunca toca Mongoose directo.

Frontend: arquitectura modular basada en features (Feature Module Architecture) — cada dominio de negocio (auth, courses, live-session, ai-assistant...) es independiente y puede cargarse de forma perezosa (lazy loading).

Patrones de diseño aplicados

Patrón	Dónde / Para qué
MVC + capa de servicios	Controller solo HTTP; Service tiene la lógica de negocio
Repository Pattern	repositories/ aísla Mongoose del resto del código
Strategy Pattern	services/ai/providers/ — interfaz IAssistantProvider (hoy: GroqProvider)
Factory Pattern	Contexto de bot por mentor (config/entrenamiento propio)
Observer / Event-driven	Socket.io: mensaje_enviado, mentor_desconectado, bot_activado
Middleware chain (RBAC)	middlewares/role.middleware.js — control de acceso por rol
Feature Modules	Frontend: cada dominio en features/ es independiente

Estructura de carpetas — Backend

```
backend/
├── src/
│   ├── config/ # conexión DB (database.js), variables de entorno
│   ├── models/ # Mongoose schemas (Usuario, Curso, ...)
│   ├── controllers/ # solo reciben req/res, delegan a services
│   ├── services/
│   │   ├── auth.service.js
│   │   ├── ai/
│   │   │   ├── rag.service.js # (pendiente)
│   │   │   ├── embedding.service.js # (pendiente)
│   │   │   └── providers/groq.provider.js # (pendiente)
│   │   ├── repositories/ # acceso a datos (pendiente)
│   │   └── middlewares/
│   │       ├── auth.middleware.js # verificarToken
│   │       ├── role.middleware.js # verificarRol
│   │       ├── validar.middleware.js # validación genérica con Zod
│   │       └── error.middleware.js # manejarErrores (centralizado)
│   │   └── validators/
│   │       ├── auth.validator.js # esquemaRegistro, esquemaInicioSesion
│   │       └── routes/
│   │           ├── auth.routes.js
│   │           └── utils/
│   │               ├── AppError.js
│   │               ├── server.js # entry point (ESM, "type": "module")
│   │               ├── scripts/check-setup.js # valida variables de entorno
│   │               ├── bruno/MentorSync-AI-API/ # colección de pruebas Bruno
│   │               └── .env.example
```

Flujo RAG (pendiente de implementar)

- Mentor sube PDF → job asíncrono trocea el texto → @xenova/transformers genera embeddings localmente → se guardan en knowledgeChunks (MongoDB Atlas Vector Search), indexados por courseld.
- Aprendiz pregunta → se embebe la pregunta → \$vectorSearch filtrado por courseld trae los chunks relevantes → se arma el prompt → Groq (Llama) genera la respuesta.
- El filtro por courseld es obligatorio: evita que el bot de un mentor "vea" contenido de otro curso.

4. Convenciones de código (obligatorias)

- **ESM, no CommonJS:** todo el backend usa import/export (backend/package.json tiene "type": "module"). Nunca require()/module.exports.
- **Nomenclatura en español:** métodos, funciones y campos de schemas propios del proyecto van en español y camelCase (registrar, iniciarSesion, verificarToken, contraseñaHash, activo). Excepción: identificadores impuestos por una librería (req, res, next, toObject()).
- **Errores con AppError:** los servicios lanzan errores con la clase AppError (utils/AppError.js), que trae statusCode. Nunca throw new Error() genérico. El middleware error.middleware.js centraliza la respuesta.
- **Validación con Zod:** todo req.body pasa por un validador (validators/) antes de llegar al service, vía el middleware genérico validar().
- **Archivos:** kebab-case (auth.service.js). **Clases:** PascalCase (AuthService, AppError).
- **Colecciones MongoDB:** camelCase en plural, en español (usuarios, cursos). Pendiente de decidir si knowledgeChunks se traduce.
- **Roles guardados:** siempre en minúsculas y en español: "aprendiz", "mentor", "administrador".
- **Sanitización NoSQL:** express-mongo-sanitize cubre req.body, req.query y req.params sin workarounds (porque se usa Express 4, no 5).
- **Logs:** pino/pino-http, nunca console.log ni morgan.

5. Estado actual de avance

Completado y verificado

- Estructura de carpetas completa (backend + frontend) generada.
- package.json de backend y frontend con todas las dependencias reales (Express 4.21.2, Node 24.19, Angular 22, Groq, @xenova/transformers, pino, swagger, dompurify+marked, Vitest, Jest+Supertest+mongodb-memory-server).
- .gitignore configurado (node_modules, .env, dist, coverage, etc.) en la raíz del repo (un solo repositorio Git, no uno por carpeta).
- Conexión a MongoDB Atlas implementada (config/database.js) — el servidor arranca SOLO después de conectar exitosamente.
- Modelo Usuario (models/user.model.js) con campos en español (nombre, email, contraseñaHash con select:false, rol, perfilMentor, activo).
- Registro (POST /api/auth/registro) y login (POST /api/auth/inicio-sesion) completos: validación con Zod, hash con bcryptjs, JWT, mensajes de error genéricos anti-enumeración, AppError con códigos HTTP correctos (409 correo duplicado, 401 credenciales inválidas).
- Middlewares verificarToken y verificarRol (RBAC) funcionando y probados.
- Ruta de prueba protegida GET /api/auth/perfil-protegido funcionando.
- Middleware centralizado de errores (manejarErrores) montado.
- Colección de pruebas Bruno (backend/bruno/MentorSync-AI-API) con 6 requests: registro aprendiz, registro mentor, login, perfil protegido, login con credenciales inválidas, registro con correo duplicado.
- TODO EL FLUJO DE AUTENTICACIÓN VALIDADO END-TO-END CON RESPUESTAS 200/201/401/403/409 CORRECTAS.

Pendiente / próximos pasos

- Modelo Curso (Mongoose) — courses collection.
- Modelo KnowledgeChunk con índice de MongoDB Atlas Vector Search (numDimensions: 384, similarity: cosine, filter por courseId).
- Modelo ChatMessage y LiveSession.
- Repository layer (repositories/) — actualmente los services acceden a los modelos directo, falta la capa de abstracción formal.
- Endpoint de subida de PDF (multer + pdf-parse) + job asíncrono de chunking.
- embedding.service.js con @xenova/transformers.
- groq.provider.js implementando IAssistantProvider (Strategy Pattern).
- rag.service.js que orquesta: embed pregunta → \$vectorSearch → prompt → Groq.
- Chat en vivo con Socket.io (eventos: mensaje_enviado, mentor_desconectado, bot_activado).
- swagger-jsdoc + swagger-ui-express montado en /api-docs (paquete ya instalado, falta implementar).
- Setup inicial de Angular 22 (esqueleto de módulos por feature: auth, courses, live-session, ai-assistant, mentor-dashboard, admin-panel).
- Componentes glass compartidos: glass-card, glass-button, glass-navbar, glass-modal, role-badge, chat-bubble.
- Tokens CSS de glassmorphism ya definidos en frontend/src/styles/_glass-tokens.scss, falta consumirlos en componentes reales.
- Integración marked + DOMPurify en el componente chat-bubble (variante bot) — la regla ya está documentada, falta implementarla.

6. Decisiones técnicas clave (con motivo)

Decisión	Motivo
Groq en vez de Gemini	Es gratis; modelos Llama rápidos vía SDK compatible con OpenAI
@xenova/transformers para embeddings	Groq no tiene endpoint de embeddings; esta librería corre local, gratis, sin API key
Express 4.21.2 en vez de Express 5	express-mongo-sanitize no funciona en Express 5 (req.query pasó a solo lectura); se evitó el workaround
ESM en todo el backend	Consistencia con Node 24.19; el código ya se escribió así desde el inicio
bcryptjs en vez de bcrypt	No requiere compilación nativa — más fácil de desplegar en Render/Railway
pino en vez de morgan	Logs estructurados en JSON, mejor para debug en producción
Nomenclatura 100% en español	Requisito explícito del estudiante para todo el proyecto
3 roles desde el diseño (no 2)	Evita refactor posterior; el admin es necesario tarde o temprano para aprobar/moderar mentores
Mensaje de error genérico en login	"Correo o contraseña incorrectos" es igual si el usuario no existe o si la contraseña es incorrecta — evita

7. Nota sobre pruebas con Bruno

Se detectó un bug real en la versión de Bruno usada: ni `bru.setEnvVar()` en `script:post-response` ni el bloque `vars:post-response` guardan de forma confiable la variable de entorno automáticamente entre requests. Flujo de trabajo adoptado (funciona 100%):

- 1. Correr el request de login.
- 2. Copiar manualmente el valor de `data.token` de la respuesta.
- 3. Ir a Environments (ícono de engranaje) → environment "Local" → pegar el valor en la fila "token" → Save.
- 4. Los demás requests que usan `{{token}}` en su header Authorization ya lo resuelven bien.

La colección completa está en `backend/bruno/MentorSync-AI-API/` con 6 requests documentados (registro aprendiz/mentor, login, perfil protegido, casos de error).

8. Variables de entorno requeridas (.env)

```
PORT=4000
MONGODB_URI=<tu connection string de MongoDB Atlas>
JWT_SECRET=<cadena larga y aleatoria>
JWT_EXPIRES_IN=7d
GROQ_API_KEY=<tu api key de Groq>
```

Correr `npm run check-setup` antes de `npm run dev` valida que estas variables existan.

9. Cómo retomar el proyecto en una nueva conversación

- Adjunta este PDF y el .zip del proyecto (carpeta docs/ con 00 a 05, más el código ya escrito).
- Dile a la IA: "Este es el estado de mi proyecto MentorSync AI, sigue estas convenciones y arquitectura ya definidas, ayúdame a continuar con [lo que sigue]."
- El siguiente paso natural es: Modelo de Curso → endpoint de subida de PDF → servicio RAG completo.

- docs/05_PROGRESS.md dentro del proyecto tiene la bitácora detallada — manténla actualizada con cada avance.

Documento generado automáticamente a partir del historial de desarrollo del proyecto MentorSync AI.